

Draw It or Lose It — Memory and Storage Management

The Gaming Room’s application, Draw It or Lose It, rotates about 200 high-definition images (≈ 8 MB each) across Android, iOS, Windows, macOS, and the web. In this paper I separate short-term memory—random-access memory (RAM)—from long-term storage on disk or in the cloud, and explain how I’ll manage both so the game stays responsive as the library grows.

Memory Management

RAM holds the data the program is using right now. To avoid out-of-memory (OOM) errors and frame drops, I decode each picture to the size I will actually draw on the screen rather than loading the original full resolution. Android’s BitmapFactory options, Apple’s Image I/O tools, and Windows decode-pixel application programming interfaces (APIs) all let me request a smaller image before it hits the display (Android Developers, 2024; Apple, 2018; Microsoft, 2025). For smooth play, I am targeting about **60 frames per second** and an **image decode under 120 ms** on mid-range phones and laptops.

I keep only the **previous, current, and next** images in a least-recently-used (LRU) cache. I start around **10% of the app heap** (the application’s available RAM pool) and watch for garbage-collection (GC) spikes during round changes; if QA sees stutter on older Android devices, I’ll **drop toward 7%** and prefetch more aggressively. When the operating system reports low memory, I evict cached items immediately (Android Developers, 2024).

All file input/output (I/O) and image decoding run on background threads so the UI thread stays free. I prefetch the next clue while the current one is showing and cancel work if the user advances quickly. Choosing memory-efficient pixel formats and, when

supported, graphics processing unit (GPU) texture compression—such as block compression (BC) on Windows or Ericsson Texture Compression 2 (ETC2) and Adaptive Scalable Texture Compression (ASTC) on mobile—reduces both RAM use and bandwidth (Microsoft, 2021). Finally, I clear references as soon as an image leaves the screen so long-lived singletons or static fields don’t accidentally retain large bitmaps; this allows GC to reclaim RAM promptly (Microsoft, 2025).

Storage Management

Storage keeps the full image catalog and its metadata. I first estimate storage needs: **200 × 8 MB ≈ 1.6 GB** for masters. Adding **10–20%** for thumbnails, manifests, and on-device caches brings the estimate to **≈1.8–2.0 GB**. On the server side, I would provision **3–5 GB** to leave room for growth and seasonal content.

To cut transfer time and post-decode memory, the build pipeline keeps high-quality masters and also produces **display-sized derivatives** (JPEG/WebP/AVIF). Because each clue is fully revealed at the **30-second** mark, serving display-sized files helps ensure the **next image is already local** by the time the current one finishes. I host assets in object storage—such as Amazon Simple Storage Service (S3), Google Cloud Storage, or Azure Blob Storage—behind a content delivery network (CDN) using **content-hashed, immutable filenames** so clients can cache safely across app launches.

On each device, I keep a **small disk cache** of recent images in the platform’s cache directory and purge it when free space is low. Lightweight metadata (IDs, tags, difficulty) lives in a compact database or JavaScript Object Notation (JSON) file. All assets travel over Hypertext Transfer Protocol Secure (HTTPS), are validated with a checksum such

as **SHA-256**, and can refresh in the background so new image packs appear without a full app update.

Comparison

RAM powers the **second-to-second** experience: decoding, drawing, and animating the current clue. Storage—local or cloud—provides the **entire catalog** and supports quick retrieval and updates. Because RAM is limited per app while storage is larger but slower, I fetch from storage **just in time**, decode only to the size I need, keep a **tiny working set** in RAM (previous/current/next), and release memory immediately after use. This approach meets the requirement to display rapidly rendered clues while allowing the library to expand.

References

Android Developers. (2024). Loading large bitmaps efficiently.

<https://developer.android.com/topic/performance/graphics/load-bitmap>

Apple. (2018). iOS memory deep dive (WWDC18 Session 416).

<https://developer.apple.com/videos/play/wwdc2018/416/>

Microsoft. (2025). Optimize animations, media, and images (UWP).

<https://learn.microsoft.com/en-us/windows/uwp/debug-test-perf/optimize-animations-and-media>

Microsoft. (2021). Block compression. [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/windows/win32/direct2d/block-compression)

[us/windows/win32/direct2d/block-compression](https://learn.microsoft.com/en-us/windows/win32/direct2d/block-compression)